

CHALLENGE #17: SSTI1 Write-up.

SSTI1

Easy

Web Exploitation

picoCTF 2025

browser_webshell_solvable

AUTHOR: VENAX

Description

I made a cool website where you can announce whatever you want! Try it out!

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is:

NOT_RUNNING

Launch Instance

Hints

1

Server Side Template Injection

50,347 users solved

95% Liked

picoCTF{FLAG}

Submit Flag

Solving Process:

The page contained a simple input field that allowed users to type a message which would then be **announced on the page**. The challenge description indicated that the task was related to **Server-Side Template Injection (SSTI)**, which occurs when user input is directly inserted into a server-side template and executed without proper validation.

← → ↺

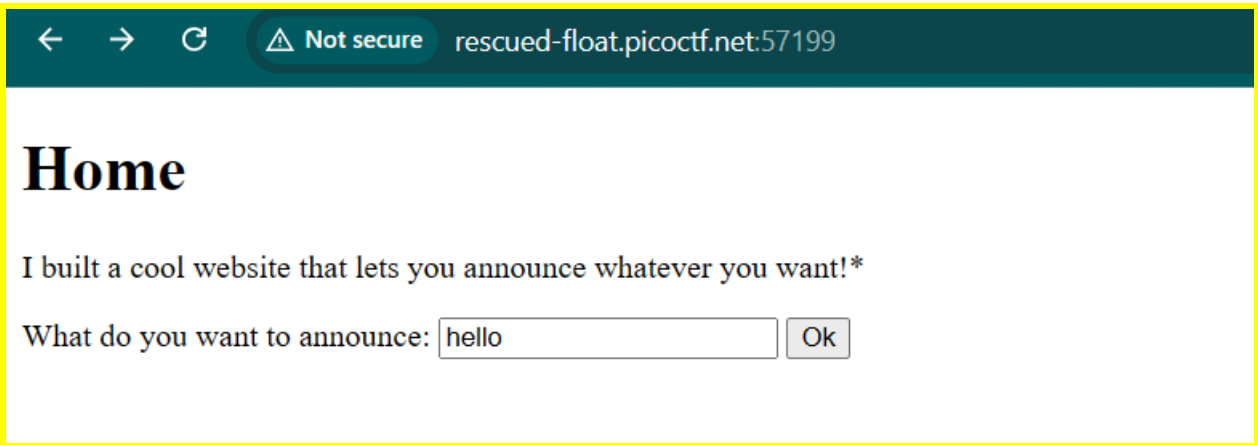
Not secure rescued-float.picoctf.net:57199

Home

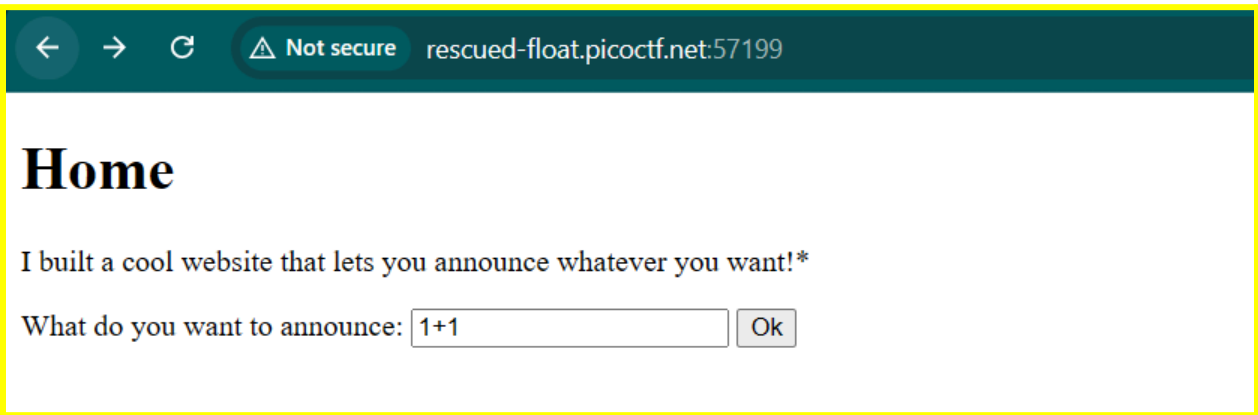
I built a cool website that lets you announce whatever you want!*

What do you want to announce:

To understand how the application behaved, a simple input such as “**hello**” was entered into the input field. After submitting it, the page displayed the same message as an announcement, confirming that the website was processing user input normally.

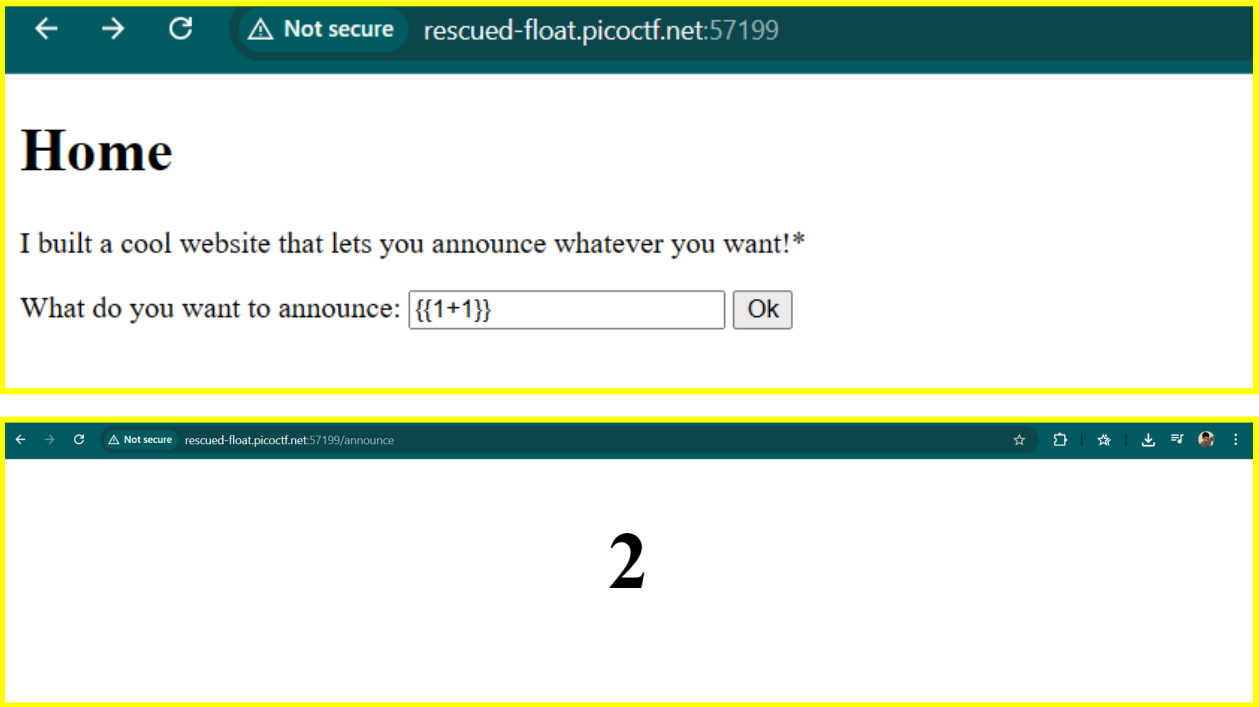


Next, a basic arithmetic expression such as **1+1** was entered to observe whether the server would evaluate the expression or simply display it as text. The output displayed “**1+1**” exactly as typed, indicating that the input was not automatically executed.

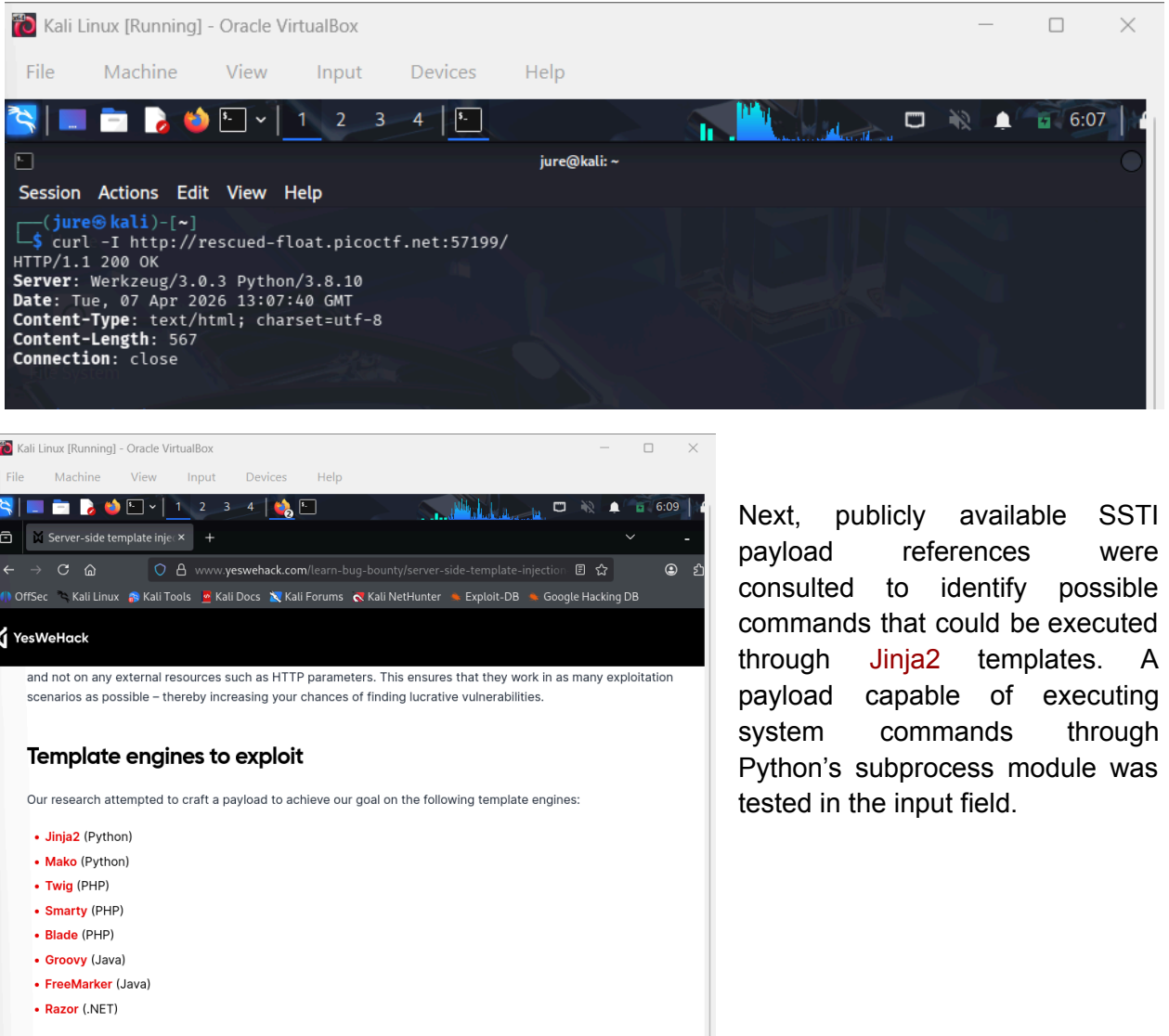


A common technique to test for SSTI vulnerabilities is to wrap expressions in **double curly braces**. Therefore, the expression **{{1+1}}** was entered into the input field and submitted. This time, the output displayed **2**, confirming that the server executed the expression inside the

template. This behavior indicated that the website was vulnerable to Server-Side Template Injection.



To determine the backend technology used by the website, the URL was copied and inspected using the **curl** command with the **-I** option to view the response headers. The headers revealed that the website was running on a Python-based web server, suggesting that the template engine might be **Jinja2**, which is commonly used in Python web frameworks.



Next, publicly available SSTI payload references were consulted to identify possible commands that could be executed through **Jinja2** templates. A payload capable of executing system commands through Python's subprocess module was tested in the input field.

Kali Linux [Running] - Oracle VirtualBox

FileMachineViewInputDevicesHelp

1234

Server-side template injection

Jinja — Jinja Documentati

www.yeswehack.com/learn-bug-bounty/server-side-template-injection

Kali LinuxKali ToolsKali DocsKali ForumsKali NetHunterExploit-DBGoogle Hacking DB

YesWeHack

Jinja2, the default template engine in Flask, is extremely powerful as it permits the execution of pure Python code.

To write a simple string, we can write this payload (note: index values may vary):

```
1 {{self.__init__.__globals__.__str__()[1786:1788]}}
```

This payload leverages the dictionary `self.__init__.__globals__`, which covers the complete global namespace of the method's module. The method `__str__` converts these globals into a string. By extracting characters at positions 1786 to 1788 from the resulting string, the payload ultimately produces the string "id".

We can now leverage this technique to build a payload that performs a remote code execution and runs system command `id`:

```
1 {{self._TemplateReference__context.cycler.__init__.__globals__.os.popen(self.__init__.
```

CYBSEC(1695) HackpicoCTF - pALT4/BATO LFI ExploitsCYBSEC: CHAll labs [W]CHALLENGE(2) FacebookSSTII

rescued-float.picocftf.net:57199

Home

I built a cool website that lets you announce whatever you want!*

What do you want to announce: {{self._TemplateReference__context.cycler.__init__.__globals__.os.popen(id).read()}}Ok

```
(jure@kali)-[~]
$ {{self._TemplateReference__context.cycler.__init__.__globals__.os.popen(id).read()}}
```

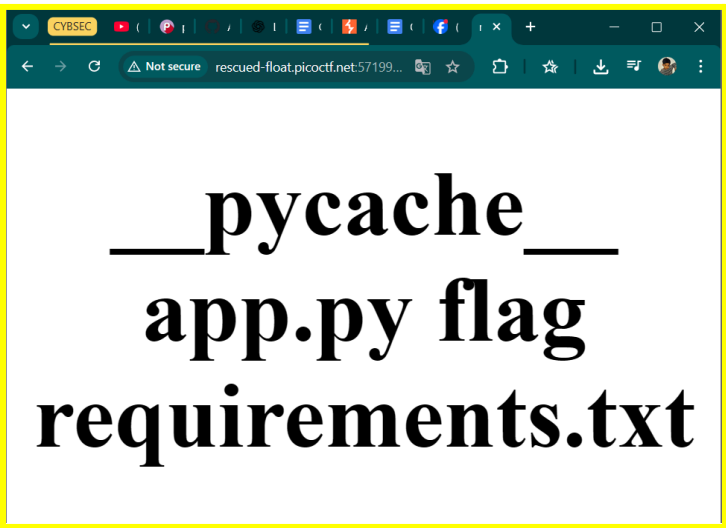
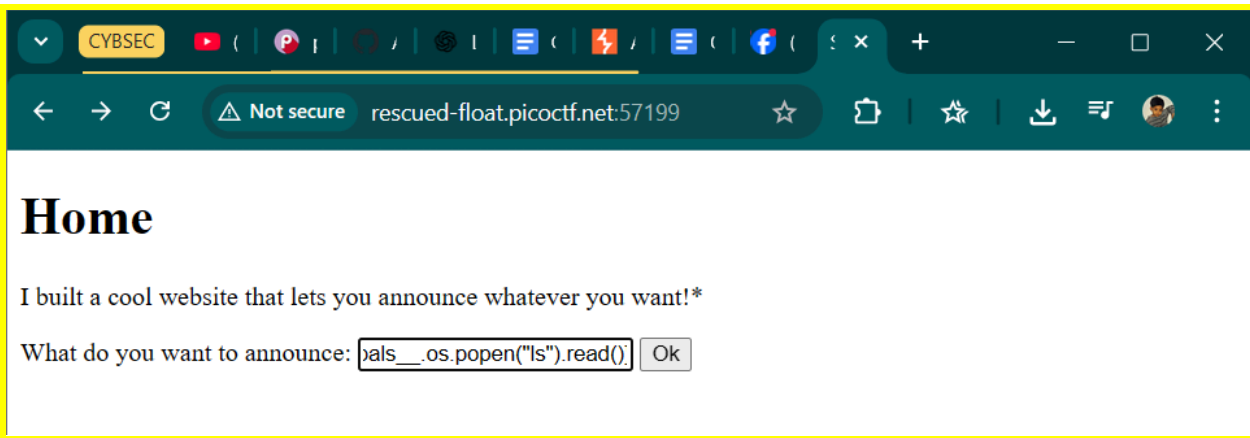
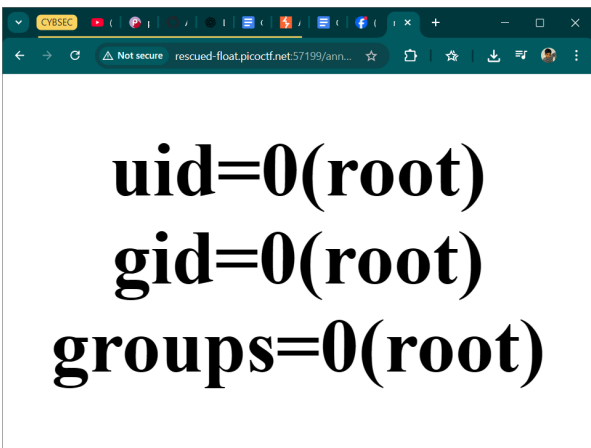
CYBSEC

rescued-float.picocftf.net:57199/ann...

Internal Server Error

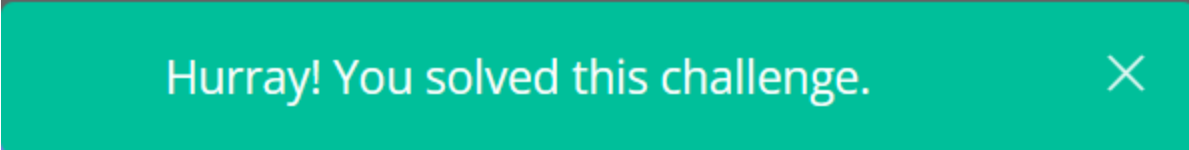
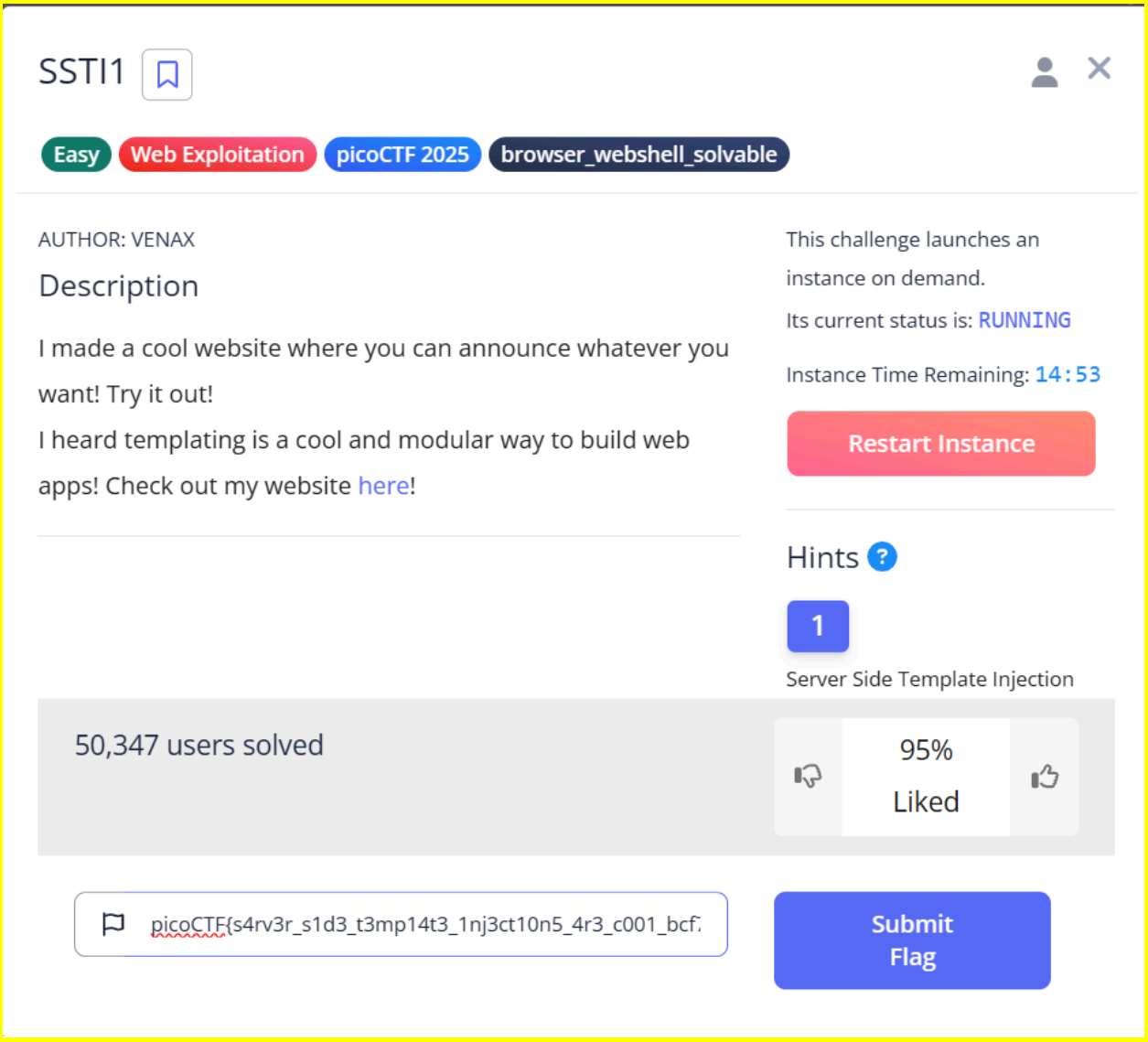
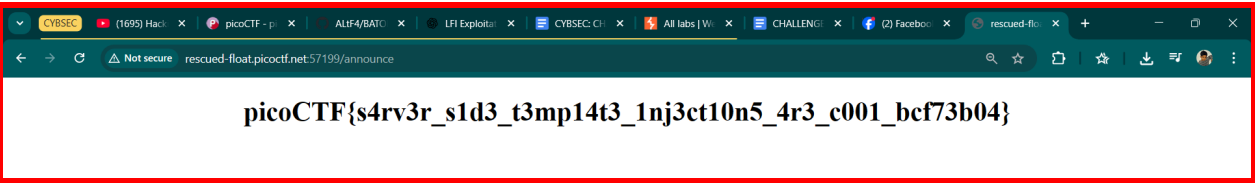
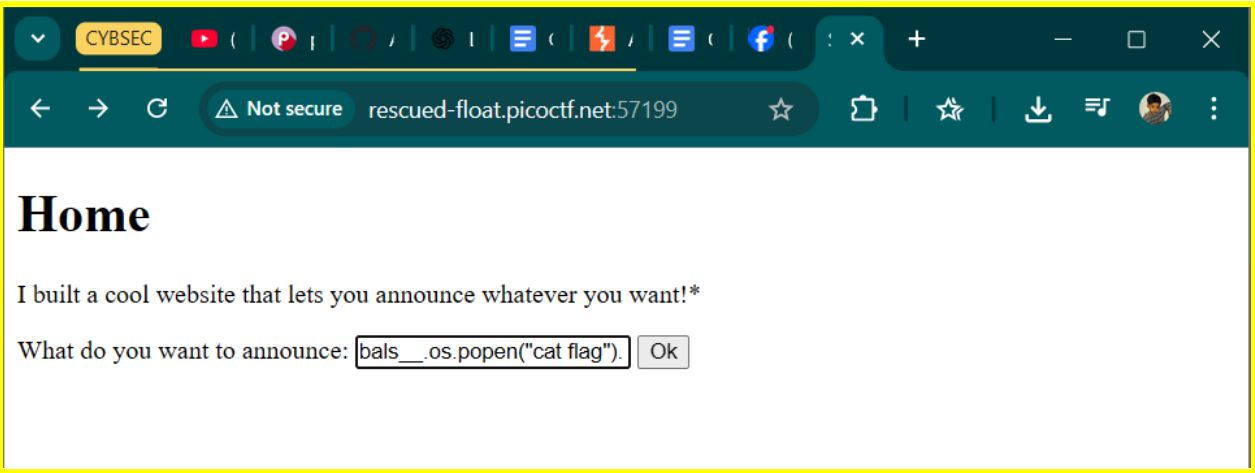
The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application.

```
(jure@kali)-[~]  
$ {{self._TemplateReference__context.cycler.__init__.__globals__.__os.popen("__id").read()}}
```



After experimenting with the payload, a command was successfully executed that listed the files in the server's directory using the **ls** command. The output revealed several files, including one named **flag**.

To retrieve the contents of the file, the payload was modified to execute the **cat** command on the flag file. Once the command was submitted, the server returned the contents of the file, revealing the flag.



Reflection:

This challenge demonstrated how improper handling of user input in server-side templates can lead to severe security vulnerabilities such as remote code execution. By exploiting SSTI, it was possible to execute system commands directly on the server and access sensitive files. The activity highlighted the importance of proper input validation and secure template handling in web application development.